

4. Program Analysis

4. Program Analysis

1. Board-Level File Structure
2. Class Inheritance Relationship
3. config.h — Pin Mapping
4. esp_box_lite_board.cc — Core Adaptation Code
 - 4.1 Constructor and Initialization Order
 - 4.2 InitializeI2c() — I2C Bus
 - 4.3 InitializeSsd1306Display() — Display
 - 4.4 InitializeButtons() — Buttons
 - 4.5 GetLed() — LED
 - 4.6 GetAudioCodec() — Audio Codec
 - 4.7 GetDisplay() — Return Display Object
 - 4.8 Three Interfaces Summary
5. Startup Flow
6. Adapting to Custom Hardware

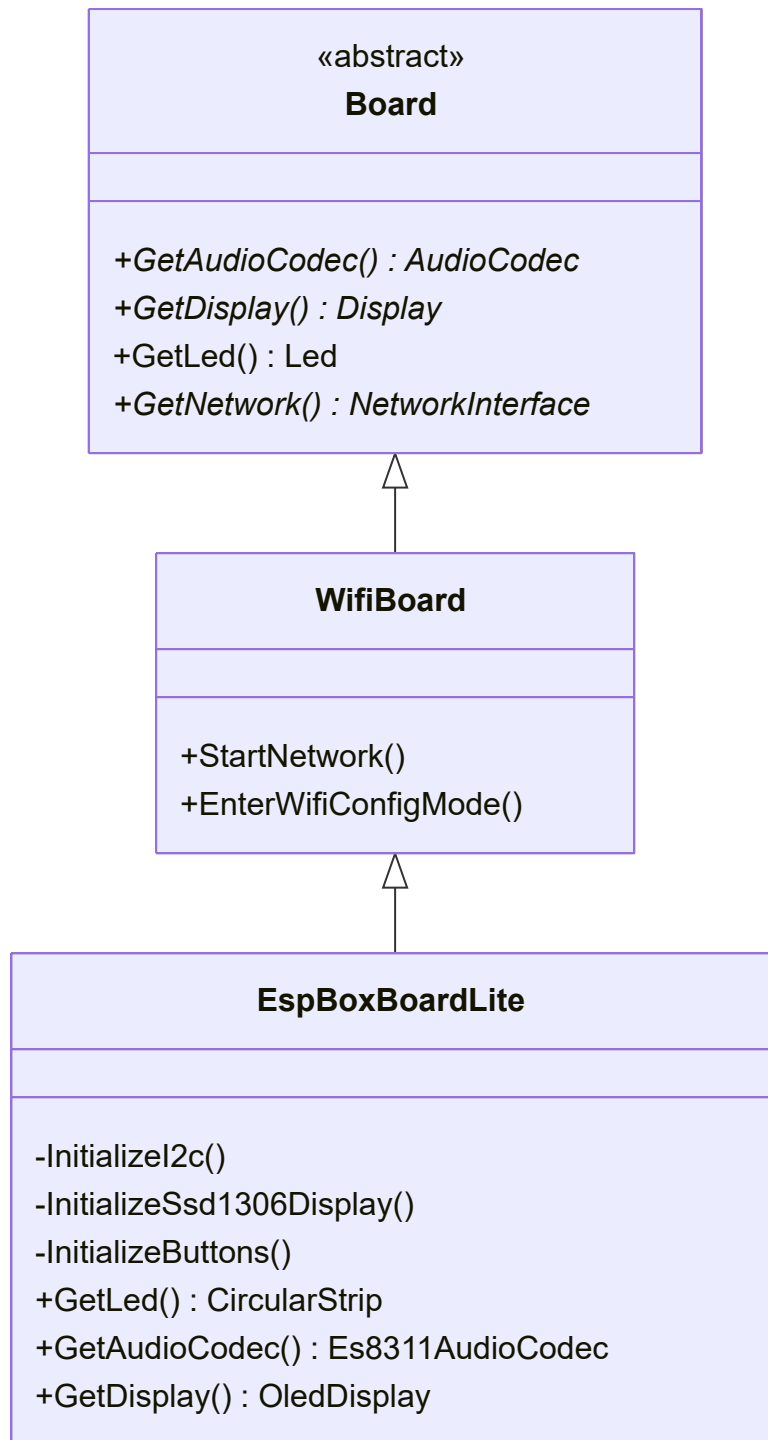
1. Board-Level File Structure

The board-level directory for `esp-box-lite` is at `main/boards/esp-box-lite/`, with 4 files:

```
main/boards/esp-box-lite/  
├─ config.h           ← Pin and parameter macro definitions  
├─ esp_box_lite_board.cc ← Board-level initialization (core)  
├─ box_audio_codec_lite.h ← Audio codec wrapper  
└─ config.json        ← Build configuration
```

2. Class Inheritance Relationship

The framework isolates hardware differences through three layers of inheritance: abstract base class `Board` defines peripheral interfaces, `wifiBoard` adds Wi-Fi networking capability, and `EspBoxBoardLite` interfaces with specific hardware.



Layer	Class	Responsibility
Abstract base class	<code>Board</code>	Defines pure virtual functions <code>GetAudioCodec()</code> , <code>GetDisplay()</code> , <code>GetNetwork()</code> , etc.; framework only recognizes these interfaces
Wi-Fi board	<code>wifiBoard</code>	Implements Wi-Fi networking, provisioning mode, network status callbacks
Specific board	<code>EspBoxBoardLite</code>	Adds this board's I2C / SSD1306 / ES8311 / buttons

`Board::GetInstance()` creates an instance via `DECLARE_BOARD(EspBoxBoardLite)` macro, expanded as:

```
void* create_board() {
    return new EspBoxBoardLite();
}
```

Framework calls `create_board()` to get the board-level object, then accesses peripherals through `Board` base class pointer. Adding a new board model only requires writing your own subclass + one line of `DECLARE_BOARD`, with zero framework code changes.

3. config.h — Pin Mapping

`config.h` is the first step in board-level adaptation, centrally defining all hardware-connected GPIOs as macros. All initialization functions in `.cc` reference these macros instead of hardcoded numbers, so changing pins only requires modifying this one file.

```
// ===== Audio I2S pins =====
#define AUDIO_I2S_GPIO_MCLK GPIO_NUM_15 // Master clock
#define AUDIO_I2S_GPIO_WS GPIO_NUM_18 // word select
#define AUDIO_I2S_GPIO_BCLK GPIO_NUM_16 // Bit clock
#define AUDIO_I2S_GPIO_DIN GPIO_NUM_17 // Data in (MIC)
#define AUDIO_I2S_GPIO_DOUT GPIO_NUM_8 // Data out (SPK)

// ===== Audio I2C control =====
#define AUDIO_CODEC_I2C_SDA_PIN GPIO_NUM_41 // ES8311 I2C SDA
#define AUDIO_CODEC_I2C_SCL_PIN GPIO_NUM_42 // ES8311 I2C SCL
#define AUDIO_CODEC_ES8311_ADDR ES8311_CODEC_DEFAULT_ADDR // ES8311 I2C address (0x18)

// ===== PA enable =====
#define AUDIO_CODEC_PA_PIN GPIO_NUM_7 // NS4150 PA enable

// ===== Onboard peripherals =====
#define BUILTIN_LED_GPIO GPIO_NUM_48 // WS2812 RGB LED
#define BOOT_BUTTON_GPIO GPIO_NUM_0 // BOOT button

// ===== Display =====
#define DISPLAY_WIDTH 128
#define DISPLAY_HEIGHT 32
#define DISPLAY_MIRROR_X true
#define DISPLAY_MIRROR_Y true
```

4. esp_box_lite_board.cc — Core Adaptation Code

4.1 Constructor and Initialization Order

Constructor initializes three major peripherals in dependency order: I2C bus is the communication foundation for all I2C devices and must be created first; screen is hung on I2C and initialized second; buttons are independent of the bus and bound callbacks last.

```

EspBoxBoardLite() : boot_button_(BOOT_BUTTON_GPIO) {
    InitializeI2c();           // ① I2C bus (shared by display + audio)
    InitializeSsd1306Display(); // ② SSD1306 display (depends on I2C)
    InitializeButtons();       // ③ Button callbacks (independent)
}

```

4.2 InitializeI2c() — I2C Bus

The SSD1306 (display) and ES8311 (audio chip) on the board share one I2C bus; this function creates the host controller and returns `i2c_bus_` handle, which both subsequent peripheral initializations pass into.

```

void InitializeI2c() {
    i2c_master_bus_config_t i2c_bus_cfg = {
        .i2c_port = (i2c_port_t)1,
        .sda_io_num = AUDIO_CODEC_I2C_SDA_PIN,    // GPIO 41
        .scl_io_num = AUDIO_CODEC_I2C_SCL_PIN,    // GPIO 42
        .clk_source = I2C_CLK_SRC_DEFAULT,        // Default clock source
        .glitch_ignore_cnt = 7,                  // Glitch filter count
        .flags = {
            .enable_internal_pullup = 1,          // Enable internal pull-up
        },
    };
    ESP_ERROR_CHECK(i2c_new_master_bus(&i2c_bus_cfg, &i2c_bus_)); // Create I2C master bus
}

```

`enable_internal_pullup = 1` enables ESP32 internal pull-up, saving external pull-up resistors.

4.3 InitializeSsd1306Display() — Display

SSD1306 is a 128×32 monochrome OLED driver IC, communicating via I2C. This function creates I2C panel IO → creates SSD1306 panel → hardware initialization → finally wraps as framework's `OledDisplay` object, stored in `display_` member for `GetDisplay()` to return.

```

void InitializeSsd1306Display() {
    // ① Create I2C panel IO
    esp_lcd_panel_io_i2c_config_t io_config = {
        .dev_addr = 0x3C,                // SSD1306 I2C address
        .control_phase_bytes = 1,        // 1-byte control phase
        .dc_bit_offset = 6,              // D/C bit at bit 6
        .lcd_cmd_bits = 8, .lcd_param_bits = 8, // Cmd/param bit width
        .scl_speed_hz = 400 * 1000,      // I2C speed 400kHz
    };
    esp_lcd_new_panel_io_i2c_v2(i2c_bus_, &io_config, &panel_io);

    // ② Create SSD1306 panel
    esp_lcd_panel_ssd1306_config_t ssd1306_config = {
        .height = static_cast<uint8_t>(DISPLAY_HEIGHT), // Display height 32px
    };
    panel_config.vendor_config = &ssd1306_config;
    esp_lcd_new_panel_ssd1306(panel_io, &panel_config, &panel);
}

```

```
// Ⓢ Reset → Init → Turn on
esp_lcd_panel_reset(panel);
esp_lcd_panel_init(panel);
esp_lcd_panel_disp_on_off(panel, true);

// Ⓢ Wrap as OledDisplay (framework Display interface)
display_ = new OledDisplay(panel_io, panel, DISPLAY_WIDTH, DISPLAY_HEIGHT,
                           DISPLAY_MIRROR_X, DISPLAY_MIRROR_Y);

}
```

Display Interface — Wrapped `OledDisplay` inherits from `Display`; framework controls screen through the following virtual functions, completely decoupled from underlying driver IC:

```
class Display {
public:
    virtual void SetStatus(const char* status);           // Set status bar text
    virtual void ShowNotification(const char* text);      // Show notification (e.g., WiFi
config)
    virtual void SetEmotion(const char* emotion);         // Switch emotion icon
    virtual void SetChatMessage(const char* role, const char* content); // Set chat message
    virtual void SetTheme(Theme* theme);                 // Switch theme (light/dark)
    virtual void SetupUI();                               // Initialize UI layout
    // ...
};
```

In other words, this initialization code is only responsible for "turning on the screen"; after that, the framework controls what displays on the screen through interfaces like `SetStatus()`, `SetEmotion()`, `SetChatMessage()`, no longer caring whether the screen is SSD1306 or ST7789.

4.4 InitializeButtons() — Buttons

`boot_button_` is essentially a wrapper for `BOOT_BUTTON_GPIO` (GPIO0); GPIO configuration and debouncing are completed by passing the pin number in the constructor. Here you only need to register callbacks: pressing BOOT during startup enters provisioning mode, pressing BOOT during normal operation wakes/interrupts dialogue.

```
void InitializeButtons() {
    boot_button_.OnClick([this]() {
        auto& app = Application::GetInstance();
        if (app.GetDeviceState() == kDeviceStateStarting) {
            EnterWifiConfigMode();           // Startup → Enter WiFi config
            return;
        }
        app.ToggleChatState();               // Running → Toggle chat state
    });
}
```

The `Button` class (located in `main/boards/common/button.h`) is a C++ wrapper for ESP-IDF's `iot_button` component, with internal debounce and multi-click detection already handled; users only need to register `OnClick` / `OnDoubleClick` / `OnLongPress` callbacks.

4.5 GetLed() — LED

LED Interface — Framework only recognizes one method `OnStateChanged()`, automatically called by the state machine on every state transition:

```
class Led {
public:
    virtual void OnStateChanged() = 0; // Called by framework on state change
};
```

`CircularStrip` is the framework's built-in WS2812 lighting effect implementation, internally reading current device state (idle/listening/speaking/provisioning, etc.) to automatically switch corresponding colors and animations.

```
virtual Led* GetLed() override {
    static CircularStrip led(BUILTIN_LED_GPIO, 1); // GPIO48, 1 WS2812 LED
    return &led;
}
```

If using a regular single-color LED connected to a regular GPIO, replace with `SingleLed(gpio)`; the framework-called `OnStateChanged()` interface remains unchanged.

4.6 GetAudioCodec() — Audio Codec

AudioCodec Interface — `Es8311AudioCodec` inherits from `AudioCodec`; framework controls audio through the following public methods, not caring about the underlying chip:

```
class AudioCodec {
public:
    virtual void SetOutputVolume(int volume); // Set speaker volume 0~100
    virtual void EnableInput(bool enable); // Enable/disable microphone
    virtual void EnableOutput(bool enable); // Enable/disable speaker
    virtual void OutputData(vector<int16_t>& data); // Output PCM data to speaker
    virtual bool InputData(vector<int16_t>& data); // Capture PCM data from microphone
    virtual void Start(); // Start audio hardware
protected:
    virtual int Read(int16_t* dest, int samples) = 0; // Low-level I2S read
    virtual int Write(const int16_t* data, int samples) = 0; // Low-level I2S write
};
```

`GetAudioCodec()` passes the 9 macros from `config.h` as parameters to the `Es8311AudioCodec` constructor, creating a ready-to-use audio object:

```
virtual AudioCodec* GetAudioCodec() override {
    static Es8311AudioCodec audio_codec(
        i2c_bus_, I2C_NUM_1, // I2C bus handle & port
        AUDIO_INPUT_SAMPLE_RATE, // Recording sample rate 16000 Hz
        AUDIO_OUTPUT_SAMPLE_RATE, // Playback sample rate 16000 Hz
        AUDIO_I2S_GPIO_MCLK, // MCLK = GPIO 15
        AUDIO_I2S_GPIO_BCLK, // BCLK = GPIO 16
    );
    return &audio_codec;
}
```

```

        AUDIO_I2S_GPIO_WS,           // WS = GPIO 18
        AUDIO_I2S_GPIO_DOUT,        // DOUT = GPIO 8 (playback/SPK)
        AUDIO_I2S_GPIO_DIN,        // DIN = GPIO 17 (recording/MIC)
        AUDIO_CODEC_PA_PIN,         // PA_EN = GPIO 7
        AUDIO_CODEC_ES8311_ADDR);   // ES8311 I2C address 0x18
    return &audio_codec;
}

```

`Es8311AudioCodec` constructor internally completes three things: ① Configure ES8311 registers to ADC+DAC duplex mode via I2C; ② Create I2S full-duplex channels (TX playback, RX recording); ③ Implement `Read()` / `write()` pure virtual functions. After framework gets `AudioCodec*`, it calls `Start()` to start audio, `EnableInput(true)` to open mic, `InputData()` to capture PCM, `OutputData()` to play PCM.

4.7 GetDisplay() — Return Display Object

```

virtual Display* GetDisplay() override {
    return display_;
}

```

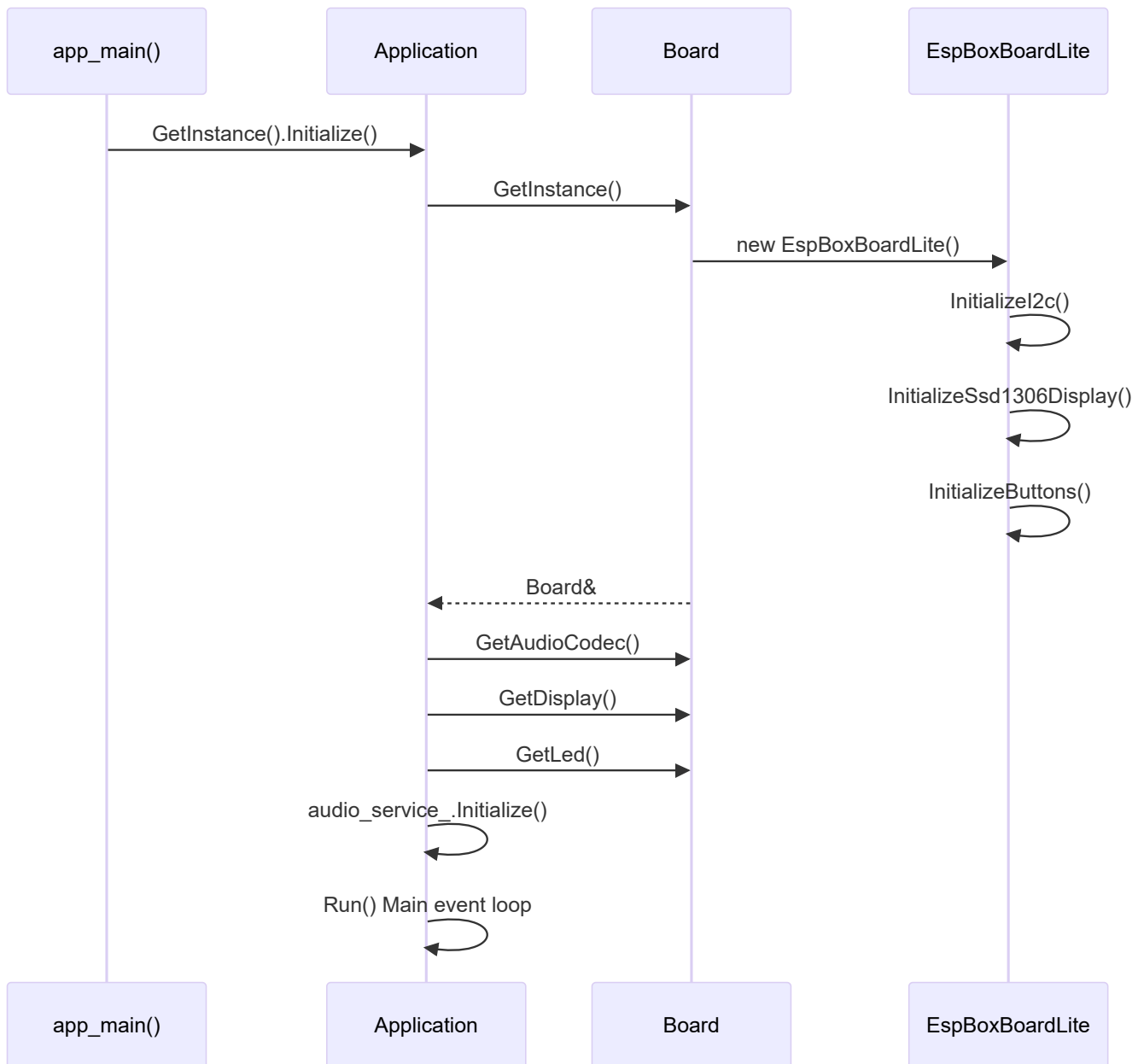
One line returns the `OledDisplay` pointer created in the constructor. After framework gets it, it calls `SetupUI()` to initialize UI layout, then controls display content through `SetStatus()` / `SetEmotion()` / `SetChatMessage()`.

4.8 Three Interfaces Summary

Peripheral	Return Type	Key Methods Called by Framework	Replace With for Different Hardware
LED	<code>Led*</code>	<code>OnStateChanged()</code>	<code>SingleLed</code> (regular GPIO LED) or custom <code>Led</code> subclass
Display	<code>Display*</code>	<code>SetupUI()</code> , <code>SetStatus()</code> , <code>SetEmotion()</code> , <code>SetChatMessage()</code>	Change constructor's 4 steps after changing display driver IC, interface unchanged
Audio	<code>AudioCodec*</code>	<code>Start()</code> , <code>EnableInput()</code> , <code>EnableOutput()</code> , <code>InputData()</code> , <code>OutputData()</code> , <code>SetOutputVolume()</code>	<code>Es8388AudioCodec</code> / <code>Es8374AudioCodec</code> , etc., interface unchanged

5. Startup Flow

Complete call sequence between board-level code and framework:



`app_main()` calls `Application::GetInstance().Initialize()` to enter framework main control.

Initialization phase has two steps:

Step 1: Create board-level instance. `Board::GetInstance()` calls the `create_board()` generated by `DECLARE_BOARD` macro, `new EspBoxBoardLite()` triggers constructor, executing `InitializeI2c()` (create I2C bus), `InitializeSsd1306Display()` (turn on screen), `InitializeButtons()` (bind button callbacks) in sequence. After construction completes, return `Board&` reference to `Application`.

Step 2: Get peripherals. `Application` gets three peripheral objects through `GetAudioCodec()`, `GetDisplay()`, `GetLed()`, passes them to `audio_service_` to complete audio pipeline initialization. Then enters `Run()` main event loop, waiting for Wi-Fi connection, wake word detection, VAD, and other events.

The entire process has board-level code executing only once during `GetInstance()` phase; after that, framework operates peripherals through base class pointer, never touching subclass details.

6. Adapting to Custom Hardware

Step	Operation	Description
1	Copy <code>esp-box-lite/</code> → new board name	Keep file structure consistent
2	Modify <code>config.h</code>	Change to your pin numbers, screen resolution, audio chip I2C address
3	Modify <code>config.json</code>	Chip model and additional sdkconfig options
4	Replace <code>InitializeSsd1306Display()</code>	Replace with your screen driver (e.g., ST7789 / ILI9341)
5	Replace <code>GetAudioCodec()</code>	Replace with your audio chip (e.g., ES8388 / ES8374)
6	Change <code>DECLARE_BOARD(YourBoard)</code>	Register board class, takes effect at compile

`main/boards/` has 70+ board references; find one with closest hardware to copy from.